

おぼえがき

テーブル定義書

設計書 v6 / 7. データ設計

システム名	おぼえがき（接客業・営業職向け 会話記録アプリ）
DBMS	PostgreSQL 15+（Supabase）
文字コード	UTF-8
テーブル数	6
権限管理	Row Level Security（RLS）。user_id による完全分離
命名規則	テーブル・カラムともスネークケース、複数形
主キー	すべて uuid（gen_random_uuid()）
作成者	SHINTARO KOMAKI

テーブル一覧

物理名	論理名 / 役割
users	利用者。Supabase Auth と 1:1
people	お客さん。記録の対象
topic_masters	話題マスタ。user_id が NULL なら初期マスタ
topics	人 × 話題。バブル1個=この1行。スコアと NG フラグを持つ
keywords	キーワードマスタ。利用者単位
records	★本体。1回の保存=1レコード。すべての集計の元

削除したテーブルの履歴

categories / fact_categories	v2 で話題に統合（判断2）
facts	v4 で records に統合（判断5-2）
conversation_logs	records に改名。内容が1種類になったため
groups	v6 で廃止（判断13）。会社名検索で代替できる
visits	v7 で廃止（判断16）。records.talked_at で代替できる

users

利用者

Supabase Auth の auth.users と 1:1。アプリ側で最小限の情報だけ持つ。

No	カラム物理名	論理名	型	NULL	既定値	キー	説明
1	id	ID	uuid	×	—	PK	auth.users.id と同一
2	email	メールアドレス	text	×	—		ログインID
3	created_at	作成日時	timestampz	×	now()		

インデックス

PRIMARY KEY (id)

備考

- Supabase Auth が発行する uuid をそのまま主キーにする
- パスワードは Supabase 側が保持。アプリのテーブルには持たない

people

お客さん

記録の対象。名前だけで登録でき、他はすべて任意。

No	カラム物理名	論理名	型	NULL	既定値	キー	説明
1	id	ID	uuid	×	gen_random_uuid()	PK	
2	user_id	利用者ID	uuid	×	—	FK	→ users.id (ON DELETE CASCADE)
3	name	名前	text	×	—		唯一の必須項目。検索対象
4	name_kana	よみがな	text	○	—		検索対象
5	age_group	年代	text	○	—		20代／30代… (列挙)
6	gender	性別	text	○	—		男性／女性／その他
7	appearance	見た目の特徴	text	○	—		「、」区切りで複数可。同姓同名の識別用。検索対象
8	company	会社名	text	○	—		検索対象・サジェスト対象 (判断14)
9	position	役職	text	○	—		表示のみ。検索対象に含めない (判断14)
10	group_id	グループID	int	○	1		v6で廃止。カラムのみ残す (判断13)
11	last_talked_at	最終会話日	date	○	—		一覧の並び順に使う。性能のための意図的な冗長カラム (判断16)
12	created_at	作成日時	timestampzt	×	now()		

インデックス

```
CREATE INDEX idx_people_user ON people(user_id);
CREATE INDEX idx_people_last ON people(user_id, last_talked_at DESC);
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_people_search ON people USING gin (
  (name || ' ' || coalesce(name_kana, ' ') || ' '
    || coalesce(appearance, ' ') || ' ' || coalesce(company, ' ')) gin_trgm_ops);
```

備考

- score のカラムは持たない。点数が付くのは話題であって人ではない (判断11)
- 写真のカラムも持たない (判断12)。PWA では保存先がきれいに解決できないため
- 検索は name / name_kana / appearance / company の4項目を OR 条件で横断する
- position は検索対象に含めない。「営業部長」で探す場面がないため
- appearance は「、」区切りで複数入力できる。分割して別テーブルにはしない (集計しないため)
- 一覧では区切って個別バッジ表示、最大2行。溢れた分は「+N」で省略。詳細画面ではすべて表示
- バッジ単体を「…」で切らない。丸ごと表示するか +N に含めるかの二択
- 分割時は複数の区切り文字を許容する： /[, , , ・]/

- last_talked_at は records から導出できるが、一覧の並び順に使うため意図的に持つ（判断16）
- 持たない場合、300人分のサブクエリが検索のたびに走る。更新は保存時の1回だけなのでズレる余地も小さい

topic_masters

話題マスタ

話題の名前だけを持つ。スコアは topics 側。

No	カラム物理名	論理名	型	NULL	既定値	キー	説明
1	id	ID	uuid	×	gen_random_uuid()	PK	
2	user_id	利用者ID	uuid	○	—	FK	NULLなら初期マスタ。→ users.id (CASCADE)
3	name	話題名	text	×	—		家族／仕事／暮らし／食べ物…
4	sort_order	並び順	int	×	0		初期マスタの表示順。記録画面チップのフォールバックに使う
5	created_at	作成日時	timestampz	×	now()		

インデックス

```
CREATE UNIQUE INDEX idx_topic_master_uniq
  ON topic_masters(coalesce(user_id::text,'global'), name);
```

備考

- ・初期マスタ10件は user_id IS NULL で投入する
- ・初期マスタの並び順：家族／食べ物／暮らし／仕事／旅行／健康・美容／趣味／エンタメ／ペット／スポーツ
- ・上ほど誰にでも当てはまり、下ほど当たれば強いが人を選ぶ、という基準で並べた
- ・「天気」と「その他」は入れない。前者は必ず下位に沈み杓を無駄にし、後者は全部そこに逃げるため
- ・利用者が新しい話題を作ると、その利用者の user_id 付きで追加される

topics

人 × 話題（バブル1個）

バブル1個＝このテーブルの1行。スコアと NG フラグを持つ。

No	カラム物理名	論理名	型	NULL	既定値	キー	説明
1	id	ID	uuid	×	gen_random_uuid()	PK	
2	person_id	お客さんID	uuid	×	—	FK	→ people.id（ON DELETE CASCADE）
3	topic_master_id	話題マスタID	uuid	×	—	FK	→ topic_masters.id（RESTRICT）
4	score	盛り上がり	numeric(5,2)	×	0		バブルの大きさ。保存のたび再計算（7.5）
5	is_ng	避ける話題	boolean	×	false		点数と別に持つ（判断6）。理由は保存しない
6	last_talked_at	最終会話日	date	○	—		時間減衰の計算に使う
7	created_at	作成日時	timestampzt	×	now()		

インデックス

CREATE UNIQUE INDEX idx_topics_uniq ON topics(person_id, topic_master_id);

CREATE INDEX idx_topics_bubble ON topics(person_id, is_ng, score DESC);

備考

- 田中さんの「食べ物」と佐藤さんの「食べ物」は別行になる
- バブルは is_ng = false のものをスコア順に10件取得する。11位以下は自動的に圏外へ落ちる
- is_ng を score と分ける理由：「反応が薄い（20点）」と「触れてはいけない」は意味が違う。0点でも“出さない”にはならない
- NG の理由は保存しない。要配慮個人情報になり得るため（判断8）
- score も records から導出できるが、バブル表示のたびに全レコードを集計するのは重いため持つ（判断16）

keywords

キーワードマスタ

ラーメン／沖縄など、話題の下にぶら下がる具体。利用者単位。

No	カラム物理名	論理名	型	NULL	既定値	キー	説明
1	id	ID	uuid	×	gen_random_uuid()	PK	
2	user_id	利用者ID	uuid	×	—	FK	→ users.id (ON DELETE CASCADE)
3	name	キーワード名	text	×	—		ラーメン／沖縄／大谷翔平…
4	created_at	作成日時	timestampz	×	now()		

インデックス

```
CREATE UNIQUE INDEX idx_keywords_uniq ON keywords(user_id, name);
CREATE INDEX idx_keywords_suggest ON keywords(user_id, name text_pattern_ops);
```

備考

- ・利用者単位で持ち、全顧客を横断してサジェストする。同じ担当者の語彙は偏るため、その方が入力が多い
- ・初回だけ手入力、2回目以降は候補から1タップ。打ち直さないでタイポも起きない
- ・表記ゆれ（大谷翔平／大谷）は顧客をまたぐと分裂しうるが、当面は許容する（判断10）

records

記録 ★本体

1回の保存＝1レコード。すべての集計の元になる。

No	カラム物理名	論理名	型	NULL	既定値	キー	説明
1	id	ID	uuid	×	gen_random_uuid()	PK	
2	person_id	お客さんID	uuid	×	—	FK	→ people.id (ON DELETE CASCADE)
3	topic_id	話題ID	uuid	×	—	FK	→ topics.id (CASCADE)。必須
4	keyword_id	キーワードID	uuid	○	—	FK	→ keywords.id (SET NULL)。任意
5	score	盛り上がり	smallint	×	—		0～100。生の入力値。必須
6	content	話した内容	text	○	—		nullable。空でも保存できる
7	talked_at	会話日	date	×	—		深夜は前日扱い（午前4時まで／M-11）
8	created_at	作成日時	timestampz	×	now()		

インデックス

```
CREATE INDEX idx_records_person ON records(person_id, talked_at DESC);
CREATE INDEX idx_records_topic ON records(topic_id, talked_at DESC);
CREATE INDEX idx_records_keyword ON records(keyword_id, talked_at DESC);
-- 情報タブは content があるものだけ引く
CREATE INDEX idx_records_content ON records(person_id, talked_at DESC)
WHERE content IS NOT NULL;
```

備考

- content が nullable であることが、この設計の要。同じ話で盛り上がったただけの日は、話題とスコアだけで保存される（判断5）
- score と talked_at を全件保存することが、唯一の「取り返しがつかない」設計判断
- これがあれば、7.5 の計算式は後からいくらでも変更できる
- 同じ話題で内容が2件のときは records が2件になる（話題・キーワード・スコアは同じ値）
- 明細に表示するのはこの score（生の値）。キーワードの計算値とは性質が違う（判断15）
- talked_at があるため visits テーブルは持たない。「今日会った人」はここから引く（判断16）
- keyword_id だけ SET NULL。CASCADE にするとキーワードを消しただけで会話の記録が消えるため

Row Level Security (RLS)

利用者ごとにデータを完全分離する。他ユーザーのデータには構造的にアクセスできない。

-- people: user_id で直接判定

```
ALTER TABLE people ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY "own rows only" ON people
FOR ALL
USING (user_id = auth.uid())
WITH CHECK (user_id = auth.uid());
```

-- records / topics: 親 (people) を辿って判定

```
ALTER TABLE records ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY "own rows only" ON records
FOR ALL
USING (EXISTS (
  SELECT 1 FROM people p
  WHERE p.id = records.person_id AND p.user_id = auth.uid()
));
```

-- topic_masters: 初期マスタ (user_id IS NULL) は全員に見せる

```
ALTER TABLE topic_masters ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY "own or global" ON topic_masters
FOR SELECT
USING (user_id = auth.uid() OR user_id IS NULL);
```

```
CREATE POLICY "own rows only" ON topic_masters
FOR INSERT WITH CHECK (user_id = auth.uid());
```

外部キーの ON DELETE

- people.user_id → users CASCADE アカウント削除で全データを落とす
- topic_masters.user_id → users CASCADE
- keywords.user_id → users CASCADE
- topics.person_id → people CASCADE お客さんを消せば話題も消える
- topics.topic_master_id → masters RESTRICT 使用中の話題名は消させない
- records.person_id → people CASCADE
- records.topic_id → topics CASCADE
- records.keyword_id → keywords SET NULL 記録は残す

扱わない情報

- アレルギー・既往歴など、施術や健康に関わる要配慮個人情報是对象外（判断8）

- ・記録するのは「会話のきっかけになる情報」だけ。人物評価ではなく事実の記録にとどめる
- ・写真も扱わない（判断12）
- ・NG話題の「理由」も保存しない